

given this project how would you begin

Thought for 13s >

Awesome project. I'd kick it off in five tight steps so you're set up to run lots of experiments quickly and fill those tables without pain. [project_3](#)

1) Set up a clean, reproducible workspace

- Colab + GPU (T4) → Runtime → Change runtime type → GPU.
- Repo layout

```
graphql
Copy code

project3/
  data/           # auto-populated by torchvision
  src/
    datasets.py
    models_mlp.py
    models_cnn.py
    train.py
    utils.py      # seeding, metrics, checkpointing, logging
  runs/          # CSV logs per run
  tables/        # final tables as CSV/Markdown
  main.py        # CLI to launch experiments
  requirements.txt
  README.md
```

- Reproducibility: set seeds for Python, NumPy, PyTorch; set `torch.backends.cudnn.deterministic=True` and `benchmark=False`.

2) Datasets, transforms, and splits

- Transforms
 - MNIST (grayscale → 1×28×28): `ToTensor()` + `Normalize(mean=0.1307, std=0.3081)`
 - CIFAR-10 (RGB 3×32×32): `ToTensor()` + `Normalize(means=(0.4914,0.4822,0.4465), stds=(0.2023,0.1994,0.2010))`
 - (Optional) For CIFAR-10 validation, allow a light augmentation toggle (random crop 32 with 4px pad, random horizontal flip) to compare w/wo aug—log which you used.
- Validation split
 - MNIST: 50k train / 10k val (holdout from original train).
 - CIFAR-10: 45k train / 5k val.
 - Use `torch.utils.data.random_split` with a fixed generator seed, and keep a small helper that returns `(train, val, test)` for each dataset so every run uses the same split.

Quick starter for loaders:

```
python
Copy code

# src/datasets.py
import torch, torchvision as tv
from torch.utils.data import DataLoader, random_split
from torchvision import transforms

MNIST_MEAN, MNIST_STD = (0.1307,), (0.3081,)
CIFAR_MEAN, CIFAR_STD = (0.4914,0.4822,0.4465), (0.2023,0.1994,0.2010)

def get_loaders(dataset="mnist", batch_size=128, augment=False, seed=42, num_work
g = torch.Generator().manual_seed(seed)
if dataset == "mnist":
    tf_train = transforms.Compose([transforms.ToTensor(), transforms.Normaliz
    tf_test = tf_train
    full = tv.datasets.MNIST(root="data", train=True, download=True, transfor
    train, val = random_split(full, [50_000, 10_000], generator=g)
    test = tv.datasets.MNIST(root="data", train=False, download=True, transfo
    in_ch, n_classes, in_shape = 1, 10, (1,28,28)
else: # cifar10
    aug_tf = [transforms.RandomCrop(32, padding=4), transforms.RandomHorizont
    tf_train = transforms.Compose(aug_tf + [transforms.ToTensor(), transforms
    tf_test = transforms.Compose([transforms.ToTensor(), transforms.Normaliz
    full = tv.datasets.CIFAR10(root="data", train=True, download=True, trans
    train, val = random_split(full, [45_000, 5_000], generator=g)
    test = tv.datasets.CIFAR10(root="data", train=False, download=True, trans
    in_ch, n_classes, in_shape = 3, 10, (3,32,32)
mk = lambda ds, shuffle: DataLoader(ds, batch_size=batch_size, shuffle=shuffl
return mk(train, True), mk(val, False), mk(test, False), in_ch, n_classes, in
```

3) Models: three MLPs + three CNNs (named, configurable)

MLPs (input is *flattened*):

- *Shallow* (1 hidden): `[784/3072] → 128 → 10`, ReLU, Dropout(*p*) .
- *Medium* (3 hidden): e.g. `[512, 256, 128]` .
- *Deep* (≥ 5 hidden): e.g. `[1024, 512, 256, 128, 64]` with dropout between most layers.

CNNs

- *Baseline* (2 conv): `Conv(32) → ReLU → MaxPool → Conv(64) → ReLU → MaxPool → FC(128) → ReLU → FC(10)` .
- *Enhanced* (BN+Dropout): add `BatchNorm` after convs and dropout (0.3–0.5) before FC.
- *Deeper* (≥ 3 conv): e.g. `Conv(32) → Conv(64) → Pool → Conv(128) → Pool → FC...` (+BN/Dropout). Keep params modest for CIFAR-10 to avoid overfitting.

4) Training loop once, reuse everywhere

- Loss: `nn.CrossEntropyLoss()` (handles softmax internally).
- Optimizers: SGD (with momentum 0.9) vs Adam.
- LR schedule (optional but helpful): StepLR or CosineAnnealing; log when used.
- Early stopping on **validation accuracy** with patience (e.g., 10 epochs).
- Log per-epoch: train loss, val loss, val acc, best acc, epoch time. Save best weights.

Minimal trainer:

```
python 📄 Copy code

# src/train.py
import torch, time
from torch import nn

def train_model(model, loaders, optimizer, epochs=50, device="cuda", patience=10):
    train_loader, val_loader = loaders
    criterion = nn.CrossEntropyLoss()
    best = {"acc": 0.0, "state": None, "epoch": 0}
    no_improve = 0
    history = []
    model.to(device)
    for epoch in range(1, epochs+1):
        t0 = time.time()
        model.train()
        tr_loss, tr_correct, n = 0.0, 0, 0
        for x,y in train_loader:
            x, y = x.to(device), y.to(device)
            optimizer.zero_grad()
            logits = model(x)
            loss = criterion(logits, y)
            loss.backward()
            optimizer.step()
            tr_loss += loss.item()*y.size(0)
            tr_correct += (logits.argmax(1)==y).sum().item()
            n += y.size(0)
        val_acc, val_loss = evaluate(model, val_loader, device, criterion)
        history.append({"epoch":epoch, "train_loss":tr_loss/n, "train_acc":tr_cor
                        "val_loss":val_loss, "val_acc":val_acc, "sec": time.time()

        if val_acc > best["acc"]:
            best.update(acc=val_acc, state=model.state_dict(), epoch=epoch)
            no_improve = 0
        else:
            no_improve += 1
            if no_improve >= patience: break
        if best["state"] is not None:
            model.load_state_dict(best["state"])
    return model, history, best

@torch.no_grad()
def evaluate(model, loader, device="cuda", criterion=None):
    model.eval()
    correct, n, tot_loss = 0, 0, 0.0
    for x,y in loader:
        x, y = x.to(device), y.to(device)
        logits = model(x)
        if criterion: tot_loss += criterion(logits, y).item()*y.size(0)
        correct += (logits.argmax(1)==y).sum().item()
        n += y.size(0)
    return correct/n, (tot_loss/n if criterion else None)
```

5) Hyperparameter plan (10–12 configs per architecture, per dataset)

You don't need all 36 combos—do **small, meaningful** random search:

- **Learning rate**: `{1e-2, 1e-3, 3e-4, 1e-4}`
- **Batch size**: `{32, 64, 128}`
- **Optimizer**: `{SGD( $\eta$ =0.9), Adam}`
- **Dropout**: `{0.2, 0.5}` (MLPs: between hidden layers; CNNs: before FC)
- **Fix epochs ceiling** (e.g., 50) + early stopping.
- For $\pm$ std in the tables: repeat the *top 2* $\downarrow$ promising configs with 3 different seeds and compute mean $\pm$ std of **validation accuracy**.

Example launcher thought process:

- Start with **baseline models** and LR sweep ($1e-2$ – $1e-4$) at batch 64, Adam.
- Keep the best LR, then try {32, 64, 128} batch.
- Finally compare Adam vs SGD(+momentum).
- Add/drop dropout to check regularization effects.
- For CNNs on CIFAR-10, test *with* vs *without* light augmentation.

Choosing the final model (per dataset)

- Pick the **single best configuration** by validation accuracy.
- **Retrain from scratch** on train+val combined with the chosen hyperparameters.
- Evaluate once on test and record **Test accuracy on final model** (one number).

How to fill the required tables quickly

For each row, log to CSV: `dataset, arch_name, lr, batch, opt, dropout, val_acc, runtime_min, seed`.

At the end, `pivot/group` to produce:

Table 1: MNIST Results (example structure)

swift Copy code

Architecture	LR	Batch	Optimizer	Dropout	Val Acc (±std)	R
MLP (shallow, 1 hidden)	...	...	...	...	...	..
MLP (medium, 3 hidden)	...	...	...	...	...	..
MLP (deep, ≥5 hidden)	...	...	...	...	...	..
Test accuracy on final MLP model: ...						
CNN (baseline, 2 conv)	...					
CNN (enhanced, BN+dropout)	...					
CNN (deeper, ≥3 conv)	...					
Test accuracy on final CNN model: ...						

Do the same for CIFAR-10 (Table 2).

Sensible starter architectures

MLP input sizes

- MNIST: `in_dim = 28*28 = 784`
- CIFAR-10: `in_dim = 3*32*32 = 3072` (expect much worse than CNNs—this is part of the analysis)

Proposed

- Shallow: `in_dim → 128 → 10` (dropout optional)
- Medium: `in_dim → 512 → 256 → 128 → 10`
- Deep: `in_dim → 1024 → 512 → 256 → 128 → 64 → 10`

CNN baseline (works for both; adjust first kernel count for MNIST vs CIFAR)

python Copy code

```
# src/models_cnn.py
import torch.nn as nn
class CNN2(nn.Module):
    def __init__(self, in_ch=3, num_classes=10, base=32, enhanced=False, deeper=False):
        super().__init__()
        bn = (lambda c: nn.BatchNorm2d(c)) if enhanced else (lambda c: nn.Identity)
        self.features = nn.Sequential(
            nn.Conv2d(in_ch, base, 3, padding=1), bn(base), nn.ReLU(),
            nn.MaxPool2d(2), # 32->16
            nn.Conv2d(base, base*2, 3, padding=1), bn(base*2), nn.ReLU(),
            nn.MaxPool2d(2), # 16->8
            *( [] if not deeper else [
                nn.Conv2d(base*2, base*4, 3, padding=1), bn(base*4), nn.ReLU(),
                nn.MaxPool2d(2), # 8->4
            ])
        )
        fc_in = (base*2 if not deeper else base*4) * (7*7 if in_ch==1 else 8*8)
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Dropout(p_drop) if enhanced else nn.Identity(),
            nn.Linear(fc_in, 128), nn.ReLU(),
            nn.Linear(128, num_classes)
        )
    def forward(self, x): return self.classifier(self.features(x))
```

Analysis section pointers for your report

- **Why CNNs > MLPs** on CIFAR-10: spatial locality & shared weights; MLPs ignore structure and explode parameter count.
- **Why deep MLP can overfit MNIST**: regularization (dropout) and early stopping impact.
- **Optimizer notes**: Adam often wins early; SGD+momentum can match with tuned LR & more epochs.
- **Augmentation effects** (CIFAR-10): small but consistent boost; quantify it.
- **Runtime management**: keep epochs sm^{-1} + early stopping; use `num_workers`; log **Runtime (min)** from your history. ↓

where do i implement the number of layers in the code for the model



Great q! You make **depth** (how many hidden layers/blocks) and **width** (units/filters) configurable in your model's `__init__`, then build layers in a loop.

Here are two clean PyTorch patterns you can drop in:

1) MLP with variable hidden layers

```
python Copy code

import torch.nn as nn

class MLP(nn.Module):
    def __init__(self, in_dim: int, hidden_sizes: list[int], num_classes: int, p_drop: float = 0.5):
        super().__init__()
        layers = []
        prev = in_dim
        for h in hidden_sizes:
            # depth = len(hidden_sizes)
            layers += [nn.Linear(prev, h), nn.ReLU(inplace=True)]
            if p_drop > 0:
                layers += [nn.Dropout(p_drop)]
            prev = h
        layers += [nn.Linear(prev, num_classes)]
        self.net = nn.Sequential(*layers)

    def forward(self, x):
        x = x.view(x.size(0), -1) # flatten
        return self.net(x)

# EXAMPLES:
# 1 hidden layer of 128
mlp1 = MLP(in_dim=28*28, hidden_sizes=[128], num_classes=10)
# 3 hidden layers: 512+256+128
mlp3 = MLP(in_dim=28*28, hidden_sizes=[512,256,128], num_classes=10, p_drop=0.5)
```

- To "test number of layers," change `hidden_sizes` length: `[128]` (1 layer), `[512,256,128]` (3 layers), etc.
- To "test units," change the values (width): `256` vs `512`, etc.

2) CNN with variable number of conv blocks

```
python Copy code

import torch.nn as nn

def conv_block(in_ch, out_ch, use_bn=True):
    return nn.Sequential(
        nn.Conv2d(in_ch, out_ch, kernel_size=3, padding=1, bias=not use_bn),
        nn.BatchNorm2d(out_ch) if use_bn else nn.Identity(),
        nn.ReLU(inplace=True),
        nn.MaxPool2d(2) # halves H,W
    )

class SimpleCNN(nn.Module):
    def __init__(self, in_ch: int, num_classes: int, num_blocks: int = 2, base: int = 32, p_drop: float = 0.5):
        super().__init__()
        channels = [base * (2**i) for i in range(num_blocks)] # width schedule p
        blocks = []
        ch_in = in_ch
        for ch_out in channels:
            # depth = num_blocks
            blocks.append(conv_block(ch_in, ch_out, use_bn=True))
            ch_in = ch_out
        self.features = nn.Sequential(*blocks)

        # figure out the spatial size after pooling num_blocks times:
        # MNIST 28x28 → after 2 pools: 7x7; CIFAR-10 32x32 → after 2 pools: 8x8,
        self.gap = nn.AdaptiveAvgPool2d(1) # avoids manual max pooling
        fc_in = ch_in
        self.classifier = nn.Sequential(
            nn.Dropout(p_drop) if p_drop > 0 else nn.Identity(),
            nn.Flatten(),
            nn.Linear(fc_in, num_classes),
        )

    def forward(self, x):
        x = self.features(x)
        x = self.gap(x)
        x = self.classifier(x)
        return x

# EXAMPLES:
# 2 conv blocks, base=32 → channels 32,64
cnn2 = SimpleCNN(in_ch=1, num_classes=10, num_blocks=2, base=32, p_drop=0.5)
# 3 conv blocks, base=32 → channels 32, 64, 128
cnn3 = SimpleCNN(in_ch=3, num_classes=10, num_blocks=3, base=32, p_drop=0.5)
```

```
tf_train = transforms.Compose([transforms.ToTensor(), transforms.Normalize(MNIST_MEAN, MNIST_STD)])
```

what does this do

That line builds the preprocessing pipeline for your training set.

```
python

tf_train = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(MNIST_MEAN, MNIST_STD)
])
```

Here's what each piece does:

- `transforms.Compose(...)`
 - Chains multiple transforms so they run in order on every sample.
- `transforms.ToTensor()`
 - Converts the PIL image/NumPy array to a PyTorch tensor with shape `[C, H, W]`.
 - Scales pixel values from 0–255 integers to 0.0–1.0 floats.
- `transforms.Normalize(MNIST_MEAN, MNIST_STD)`
 - Applies channel-wise standardization: $x \leftarrow (x - \text{mean}) / \text{std}$.
 - For MNIST (grayscale), `MNIST_MEAN=(0.1307,)` and `MNIST_STD=(0.3081,)`, so you normalize the single channel using those constants (computed from the MNIST training set).

Why this matters:

- Makes features roughly zero-mean, unit-variance → **faster, more stable training**.
- Keeps train/val/test on the **same scale** (use the same normalize values everywhere).
- For MNIST, this also ensures your input tensors end up sized `[1, 28, 28]` with standardized values typically around `~[-2, 2]`.

Tip: If you add augmentations (e.g., random crop/flip for CIFAR-10), put them **before**

`Normalize`, and keep `Normalize` near the end of the transform list.

📄 📌 🔍 ↕ 🔄 ...

ok can you give me a layout for this project if we were only doing the mlp part of it?

Thought for 5s >

Totally—here's a clean, MLP-only project layout that's easy to run, sweep, and report.

Project layout (MLP-only)

```
bash

mlp_project/
├─ README.md
├─ requirements.txt
├─ main.py                # CLI to train/evaluate one run
├─ configs/
│  └─ mnist_baseline.yaml
│  └─ cifar10_baseline.yaml
├─ src/
│  └─ data/
│  │  └─ loader.py        # get_loaders(...) for MNIST/CIFAR-10
│  └─ models/
│  │  └─ mlp.py           # flexible MLP (depth/width via list)
│  └─ train.py            # train loop + early stopping
│  └─ evaluate.py         # test-set eval for the picked model
│  └─ metrics.py          # accuracy, param counts, etc.
│  └─ utils.py            # seeding, logging CSV, checkpoint I/O
├─ runs/
│  └─ logs.csv            # one row per run (append-only)
│  └─ ckpts/              # best checkpoints by run_id
├─ tables/
│  └─ results_mnist.csv   # aggregated mean/std for the report
├─ scripts/
│  └─ sweep.py            # small grid/random search over depth/width
│  └─ train.sh            # handy bash examples
```

Key files (tiny stubs)

`src/models/mlp.py`

```
python

import torch.nn as nn

class MLP(nn.Module):
    def __init__(self, in_dim: int, hidden_sizes: list[int], num_classes: int, p=0.5):
        super().__init__()
        layers = []
        prev = in_dim
        for hidden_size in hidden_sizes:
            layers.append(nn.Linear(prev, hidden_size))
            prev = hidden_size
        layers.append(nn.Linear(prev, num_classes))
        self.layers = nn.Sequential(*layers)
```

src/data/loader.py

Use the `get_loaders(...)` you already have (MNIST & CIFAR-10 with fixed train/val splits, optional aug for CIFAR only). Return:

python

Copy code

```
train_loader, val_loader, test_loader, in_ch, num_classes, in_shape
```

src/train.py

python

Copy code

```
import time, torch
from torch import nn

@torch.no_grad()
def evaluate(model, loader, device="cuda"):
    model.eval(); correct=0; n=0
    for x,y in loader:
        x,y = x.to(device), y.to(device)
        pred = model(x).argmax(1)
        correct += (pred==y).sum().item(); n += y.size(0)
    return correct / n

def train_model(model, train_loader, val_loader, *, lr=1e-3, epochs=50, patience=
model.to(device)
opt = torch.optim.Adam(model.parameters(), lr=lr)
crit = nn.CrossEntropyLoss()
best = {"acc": 0.0, "state": None, "epoch": 0}
no_imp = 0; hist=[]
for ep in range(1, epochs+1):
    t0=time.time(); model.train(); tr_loss=0; n=0
    for x,y in train_loader:
        x,y = x.to(device), y.to(device)
        opt.zero_grad()
        logits = model(x)
        loss = crit(logits, y); loss.backward(); opt.step()
        tr_loss += loss.item()*y.size(0); n += y.size(0)
    val_acc = evaluate(model, val_loader, device=device)
    hist.append({"epoch": ep, "train_loss": tr_loss/n, "val_acc": val_acc, "s
    if val_acc > best["acc"]:
        best = {"acc": val_acc, "state": model.state_dict(), "epoch": ep}
        no_imp = 0
    else:
        no_imp += 1
    if no_imp >= patience: break
    if best["state"] is not None: model.load_state_dict(best["state"])
    return model, hist, best
```

src/utils.py

python

Copy code

```
import csv, os, random, numpy as np, torch

def set_seed(seed:int=42, deterministic=True):
    random.seed(seed); np.random.seed(seed); torch.manual_seed(seed); torch.cuda.
    if deterministic:
        torch.backends.cudnn.deterministic=True; torch.backends.cudnn.benchmark=F

def log_row(csv_path, row_dict):
    os.makedirs(os.path.dirname(csv_path), exist_ok=True)
    write_header = not os.path.exists(csv_path)
    with open(csv_path, "a", newline="") as f:
        w = csv.DictWriter(f, fieldnames=row_dict.keys())
        if write_header: w.writeheader()
        w.writerow(row_dict)
```

main.py

python

Copy code

```
import argparse, os, time, torch
from src.utils import set_seed, log_row
from src.data.loader import get_loaders
from src.models.mlp import MLP
from src.train import train_model, evaluate

def parse_hidden(s: str):
    # e.g. "512,256,128" -> [512,256,128]
    return [int(x) for x in s.split(",") if x.strip()]

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--dataset", choices=["mnist","cifar10"], default="mnist")
    ap.add_argument("--hidden", type=parse_hidden, default="256,256,128")
    ap.add_argument("--dropout", type=float, default=0.5)
    ap.add_argument("--batch", type=int, default=128)
    ap.add_argument("--lr", type=float, default=1e-3)
    ap.add_argument("--epochs", type=int, default=50)
    ap.add_argument("--patience", type=int, default=10)
```

```

model = MLP(in_dim=in_dim, hidden_sizes=args.hidden, num_classes=args.num_classes, device=device)

device = "cuda" if torch.cuda.is_available() else "cpu"
t0=time.time()
model, hist, best = train_model(model, train_l, val_l, lr=args.lr, epochs=args.epochs)
runtime = (time.time()-t0)/60
test_acc = evaluate(model, test_l, device=device)

run_name = args.run_name or f"{args.dataset}_mlp_depth{len(args.hidden)}_width{args.hidden[-1]}"
log_row("runs/logs.csv", {
    "run": run_name,
    "dataset": args.dataset,
    "hidden": "-".join(map(str,args.hidden)),
    "dropout": args.dropout,
    "batch": args.batch,
    "lr": args.lr,
    "epochs": args.epochs,
    "val_best_acc": round(best['acc'],4),
    "test_acc": round(test_acc,4),
    "runtime_min": round(runtime,2),
    "seed": args.seed
})
os.makedirs("runs/ckpts", exist_ok=True)
torch.save(model.state_dict(), f"runs/ckpts/{run_name}.pt")
print(f"[{run_name}] val_best={best['acc']:.4f} @ epoch {best['epoch']} | test_acc={test_acc:.4f}")

if __name__ == "__main__":
    main()

```

configs/mnist_baseline.yaml (optional)

```

yaml

dataset: mnist
hidden: [256, 256, 128]
dropout: 0.5
batch: 128
lr: 0.001
epochs: 50
patience: 10
seeds: [1, 2, 3]

```

scripts/sweep.py (mini grid/random search)

```

python

import itertools, subprocess, sys
depth_options = [[128], [256,128], [512,256,128], [1024,512,256,128]]
lrs = [1e-3, 3e-4]
batches = [64, 128]
seeds = [1,2,3]
for hidden, lr, bs in itertools.product(depth_options, lrs, batches):
    hidden_str = "-".join(map(str, hidden))
    for seed in seeds:
        cmd = ["python", "main.py", "--dataset", "mnist", "--hidden", hidden_str, "--batch", str(bs), "--lr", str(lr), "--epochs", "50", "--patience", "10", "--seed", str(seed)]
        print(">>> ", " ".join(cmd)); subprocess.run(cmd, check=True)

```

requirements.txt

```

nginx

torch
torchvision
numpy

```

How you'd run it

- Single run (MNIST, depth=3 with widths 512→256→128):

```

bash

python main.py --dataset mnist --hidden 512,256,128 --dropout 0.5 --lr 1e-3 --batch 128 --seed 1

```

- Small sweep:

```

bash

python scripts/sweep.py

```

This layout keeps the focus on MLPs while preserving everything you need for experiments: fixed data splits, a flexible "number of layers" via `--hidden`, clean logging to CSV, checkpoints, and a simple path to aggregate results into your tables. If you want, I can also drop these stubs into a single zip or a Colab to get you running instantly.

↳ patience=args.patience

what do you mean by patience? is this batch size

Good catch—**patience is not batch size**.

- **Batch size** = how many samples you process *at once* before a weight update (e.g., 64, 128). It affects gradient noise, memory use, and speed.
- **Patience** = the **early stopping** tolerance measured in **epochs**. It's how many consecutive epochs you'll allow with **no improvement in your validation metric** (e.g., val accuracy) before you stop training.

Quick example (matching your `patience=args.patience`):

- Suppose `patience = 10`.
- You track the best validation accuracy so far.
- Each epoch: if val accuracy improves → reset a `no_improve` counter to 0; otherwise increment it.
- If `no_improve` reaches 10, you stop and restore the best model.

Why use patience?

- Prevents overfitting and saves time when the model has plateaued.
- Lets training continue through short “dry spells” so you don’t stop on a fluke epoch.

Typical settings:

- Small/fast tasks: 5–10.
- Noisy validation or longer schedules: 10–20.
- If you use a cosine/step LR schedule, consider a patience that’s long enough to see post-schedule improvements.

TL;DR: **Batch size** controls mini-batch processing; **patience** controls *when to stop* training after several unimproved **epochs**.

📄 🌟 🔍 ⬆️ ↺ ⋮

↳ `model, hist, best = train_model(model, train_l, val_l, lr=args.lr, epochs=args.epochs, patience=args.patience, device=device)`

where does this factor in batch size?

Great eye. In your call here:

```
python                                     📄 Copy code

model, hist, best = train_model(model, train_l, val_l, lr=args.lr, epochs=args.epochs, patience=args.patience, device=device)
```

Batch size isn't a parameter of `train_model`—it's already “baked into” the data loaders you pass in:

```
python                                     📄 Copy code

train_l, val_l, test_l, ... = get_loaders(dataset=args.dataset, batch_size=args.batch_size)
```

So `args.batch` → `get_loaders(... batch_size=...)` → `Dataloader(...)`.

Then inside `train_model`, the loop:

```
python                                     📄 Copy code

for x, y in train_loader:
    ...
```

uses that batch size implicitly.

Where batch size matters

- **Steps per epoch:** `steps = ceil(N_train / batch_size)`
- **Memory:** larger batch → more GPU RAM.
- **Gradient noise:** smaller batch → noisier but sometimes better generalization.
- **LR interplay:** if you increase batch a lot, you can try the *linear scaling rule* (roughly scale $LR \propto \text{batch}$).

How to change it

Just pass a different value on the CLI:

```
bash                                     📄 Copy code

python main.py --batch 64 # or 32, 128, etc.
```

(If you want different batch sizes for train vs. val/test, you can add separate args and wire them into `get_loaders` or make two loaders with different `batch_size`.)

📄 🌟 🔍 ⬆️ ↺ ⋮


```
↳ mk = lambda ds, shuffle: DataLoader(ds, batch_size=batch_size, shuffle=shuffle,
    num_workers=num_workers, pin_memory=True) return mk(train, True), mk(val, False), mk(test,
    False), in_ch, n_classes, in_shape
```

can you explain these two lines

Yep! Those two lines are just a tiny helper (“factory”) for building `DataLoader` s, and then using it to make the three loaders.

python

 Copy code

```
mk = lambda ds, shuffle: DataLoader(ds, batch_size=batch_size,
    shuffle=shuffle, num_workers=num_workers,
    pin_memory=True)

return mk(train, True), mk(val, False), mk(test, False), in_ch, n_classes, in_shape
```

What’s happening

1. `mk = lambda ...`

- Creates a short, inline function `mk(dataset, shuffle_flag) → DataLoader`.
- It captures (`closes over`) variables from the outer scope: `batch_size` and `num_workers`.
- Args used:
 - `batch_size` : how many samples per batch.
 - `shuffle : True` randomizes sample order each epoch (good for training), `False` keeps a deterministic order (good for eval).
 - `num_workers` : how many background processes load data.
 - `pin_memory=True` : if you’re using a GPU, pinned host memory speeds up CPU→GPU transfers.

2. `return ...`

- Builds and returns:
 - `mk(train, True)` → training loader (shuffled).
 - `mk(val, False)` → validation loader (not shuffled).
 - `mk(test, False)` → test loader (not shuffled).
- Plus the metadata: `in_ch` (input channels), `n_classes`, and `in_shape` (e.g., `(1,28,28)` for MNIST).

Same thing, written verbosely

python

 Copy code

```
def mk(ds, shuffle):
    return DataLoader(ds, batch_size=batch_size, shuffle=shuffle,
        num_workers=num_workers, pin_memory=True)

train_loader = mk(train, True)    # shuffle for training
val_loader   = mk(val, False)    # no shuffle for evaluation
test_loader  = mk(test, False)

return train_loader, val_loader, test_loader, in_ch, n_classes, in_shape
```

That’s it—the lambda just keeps the code compact and consistent for all three splits.

```
→ for ep in range(1, epochs+1): t0=time.time(); model.train(); tr_loss=0; n=0 for x,y in train_loader:
x,y = x.to(device), y.to(device) opt.zero_grad() logits = model(x) loss = crit(logits, y);
loss.backward(); opt.step() tr_loss += loss.item()*y.size(0); n += y.size(0) val_acc =...
```

can you explain this for loop



Here's what that training loop is doing, line by line, in plain English:

```
python Copy code

for ep in range(1, epochs+1):
    t0 = time.time()
    model.train()
    tr_loss, n = 0.0, 0

    for x, y in train_loader:
        x, y = x.to(device), y.to(device)      # move a mini-batch to GPU/CPU
        opt.zero_grad()                        # clear old gradients
        logits = model(x)                      # forward pass: fθ(x)
        loss = crit(logits, y)                  # compute batch loss (e.g., cross-e
        loss.backward()                         # backprop: ∂loss/∂θ
        opt.step()                             # gradient descent update of θ

        tr_loss += loss.item() * y.size(0)      # accumulate *sum* of losses across
        n += y.size(0)                         # count how many samples we've seen

    val_acc = evaluate(model, val_loader, device=device) # full val pass (model.
    hist.append({
        "epoch": ep,
        "train_loss": tr_loss / n,              # average train loss over the whole
        "val_acc": val_acc,
        "sec": time.time() - t0                 # wall-clock time for this epoch
    })
```

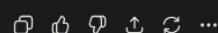
Why each piece matters

- `for ep in range(1, epochs+1)` : run multiple **epochs**; each epoch sees the whole training set once.
- `t0 = time.time()` : start a stopwatch so you can log how long the epoch took.
- `model.train()` : put layers like Dropout/BatchNorm into **training mode** (they behave differently in eval).
- `for x, y in train_loader` : iterate over **mini-batches** from your DataLoader (size = your `batch_size`).
- `x.to(device)` : put tensors on the same device as the model (GPU if available).
- `opt.zero_grad()` : gradients accumulate by default in PyTorch; zero them before each update.
- `logits = model(x)` : compute predictions.
- `loss = crit(logits, y)` : compute a scalar loss for that batch.
- `loss.backward()` : compute gradients via backprop.
- `opt.step()` : apply one optimizer step (e.g., Adam/SGD) using those gradients.
- `tr_loss += loss.item() * y.size(0)` : keep the **sum** of per-sample losses. Multiplying by `batch_size` (`y.size(0)`) converts the batch **mean** loss into a **sum** so you can later average correctly even if the last batch is smaller.
- `n += y.size(0)` : total number of samples processed this epoch.
- `val_acc = evaluate(...)` : run a clean validation pass to measure generalization (with `model.eval()` inside `evaluate`).
- `hist.append(...)` : log epoch stats—useful for plots, early stopping, and your results table.

Common gotchas to avoid

- Forgetting `model.train()` / `model.eval()` leads to misleading metrics (BN/Dropout misbehave).
- Forgetting `opt.zero_grad()` causes gradient accumulation and exploding updates.
- Averaging loss incorrectly: always divide the **sum** by total `n` (`tr_loss / n`), not by number of batches, to handle uneven last batch.

If you want, I can add comments like these directly into your training file so it's self-documenting.



↳ no_imp

what is this variable for

`no_imp` = "no improvement" counter for early stopping.

- **What it tracks:** how many **consecutive epochs** passed **without** improving the best validation metric (e.g., `val_acc`).
- **Where it's used:** compared to `patience`. If `no_imp >= patience`, training stops early.

Quick walkthrough (typical pattern):

```
python Copy code

best = {"acc": 0.0, "state": None, "epoch": 0}
no_imp = 0

for epoch in range(1, epochs+1):
    ... # train for one epoch
    val_acc = evaluate(model, val_loader)

    if val_acc > best["acc"]:
        best = {"acc": val_acc, "state": model.state_dict(), "epoch": epoch}
        no_imp = 0 # reset on improvement
    else:
        no_imp += 1 # bump when no improvement

    if no_imp >= patience: # stop if too many meh epochs
        break

# restore the best model before returning
if best["state"] is not None:
    model.load_state_dict(best["state"])
```

Why it matters: prevents wasting epochs once the model plateaus or starts overfitting, but still allows a few "dry" epochs so you don't stop on a fluke.

📄 👍 🔒 ⬆️ ↺ ...

↳ tr_loss=0; n=0

what do these two variables mean?

They're the running totals for your epoch:

- `tr_loss = 0.0`

Starts the **sum of training loss over the whole epoch**. Each batch you do:

```
python Copy code

tr_loss += loss.item() * y.size(0)
```

Since `loss` is usually the **mean** over the batch, multiplying by `y.size(0)` converts it to a **sum** so uneven last batches don't skew things.

- `n = 0`

Counts the **number of samples seen** this epoch. Each batch:

```
python Copy code

n += y.size(0)
```

At the end of the epoch you report the **average training loss** as:

```
python Copy code

avg_train_loss = tr_loss / n
```

That's the correct per-sample average across the entire epoch, regardless of batch sizes.

📄 👍 🔒 ⬆️ ↺ ...

↳ `correct += (pred==y).sum().item()`

what is going on in this line?

That line is counting how many predictions were correct in the current mini-batch and adding it to a running total:

```
python Copy code

correct += (pred == y).sum().item()
```

Breakdown:

- `pred == y` → elementwise comparison of predicted labels vs. true labels. Returns a boolean tensor like `[True, False, True, ...]` (same length as the batch).
- `(pred == y).sum()` → sums `True` values (treated as 1) to get the **number of correct samples** in this batch. Result is a 0-D tensor (a scalar tensor) on the same device as `pred/y`.
- `.item()` → converts that scalar tensor to a plain Python number (e.g., `int`) so you can add it to a Python counter.
- `correct += ...` → accumulates over batches so, after the epoch, you can compute accuracy as `correct / n` where `n` is the total number of samples seen.

Equivalent, more explicit version (and slightly safer if your tensors are on GPU):

```
python Copy code

batch_correct = (pred == y).to(torch.int64).sum().detach().cpu().item()
correct += batch_correct
```

Common gotchas:

- Ensure `pred` and `y` are the same shape (e.g., both `[batch_size]`) and dtype for labels is `torch.long (int64)`.
- If you're working on GPU, `.item()` causes a tiny sync; that's fine here since it's once per batch.

📄 🌟 🔍 ⬆️ ↺ ...

↳ `x = x.view(x.size(0), -1)`

what is this doing

That line **flattens each sample** in the batch so an MLP can take it as a single vector.

```
python Copy code

x = x.view(x.size(0), -1)
```

- Assume `x` is a batch of images with shape `[B, C, H, W]` (e.g., `[64, 1, 28, 28]`).
- `x.size(0)` is `B` (the batch size).
- `-1` tells PyTorch "infer the rest," so it becomes `C·H·W`.
- Resulting shape: `[B, C·H·W]` (e.g., `[64, 784]` for MNIST), which a `nn.Linear(in_features=C·H·W, ...)` expects.

Why do this?

- MLPs need 2D inputs `[batch, features]`.
- CNNs keep `[B, C, H, W]` and don't flatten until the classifier head (or use GAP).

Notes & gotchas:

- `view` doesn't copy memory; it **requires a contiguous tensor**. If you've used ops like `.permute` / advanced slicing, call `.contiguous()` first:
`x = x.contiguous().view(x.size(0), -1)`
- Equivalent, slightly safer alternatives:
`x = torch.flatten(x, start_dim=1)` or `x = x.reshape(x.size(0), -1)`.

Tiny example:

- Before: `x.shape == (32, 3, 32, 32)` → after: `(32, 3072)`.
Now you can feed `x` into `nn.Linear(3072, hidden)` in your MLP.

📄 🌟 🔍 ⬆️ ↺ ...

Thought for 23s >

If you're seeing `accuracy = 1.0` everywhere, that's almost always a wiring bug. Here are the most common causes and the exact checks/fixes:

Quick culprits to check (and fix)

1. Comparing to the wrong thing

- Buggy: `correct += (pred == pred).sum().item()` ← always equals batch size ⇒ `acc = 1.0`
- Correct: `correct += (pred == y).sum().item()`

2. Evaluating on the training loader by mistake

- Buggy: `evaluate(model, train_loader, ...)`
- Correct: `evaluate(model, val_loader, ...)` (and for final report `test_loader`)

3. Train/Val leakage (same samples in both)

- If you didn't split, or you reused the same dataset for both loaders, you'll "ace" validation.
- With `random_split`, verify disjointness once:

```
python
# for torch.utils.data.Subset objects from random_split
i_train = set(train.indices); i_val = set(val.indices)
assert i_train.isdisjoint(i_val), "Train/Val overlap!"
```

4. Label bug (targets equal predictions or all a single class)

- Sanity-check a batch:

```
python
x, y = next(iter(train_loader))
print("y unique:", y.unique(sorted=True))
with torch.no_grad():
    pred = model(x.to(device)).argmax(1).cpu()
    print("pred unique:", pred.unique(sorted=True))
    print("batch acc:", (pred==y).float().mean().item())
```

- If `y.unique()` shows only one value or `pred` equals `y` before training, your dataset/labels are wrong (e.g., a custom Dataset returning `(x,x)`).

5. Broadcasting accident hides a mistake

- Ensure `pred.shape == y.shape == [batch]`. If you used `keepdim=True`, fix it:

```
python
# Bad for accuracy comparison:
pred = logits.argmax(1, keepdim=True) # shape [B,1]
# Good:
pred = logits.argmax(1)                # shape [B]
```

6. Not switching to eval mode in evaluation

- Inside `evaluate`, you should have:

```
python
@torch.no_grad()
def evaluate(...):
    model.eval()
    ...
```

(And use `model.train()` during training.) This won't usually cause 1.0, but good hygiene.

Minimal correct evaluation snippet (drop-in)

```
python
@torch.no_grad()
def evaluate(model, loader, device="cuda"):
    model.eval()
    correct = 0
    n = 0
    for x, y in loader:
        x, y = x.to(device), y.to(device)
        pred = model(x).argmax(1) # [B]
        correct += (pred == y).sum().item()
        n += y.size(0)
    return correct / n
```

Two fast sanity tests

- Random baseline:** before training, run `evaluate(model, val_loader)`. For 10 classes, it should be ~0.10, not 1.0.
- Tiny overfit probe:** train on just 256 training samples. You should reach ~1.0 **train** accuracy, but **val** accuracy should be far from 1.0. If both read 1.0, something's leaking or being compared incorrectly.

If you paste your `evaluate(...)` and the line where you call it, I'll pinpoint which of the above is hitting you and provide the exact patch.

can you write a version of the getloaders function without augmenting the image?

Absolutely—here's a clean, no-augmentation version. It **always** normalizes and **never** applies flips/crops/jitter.

```
python Copy code

# src/data/loader_no_aug.py
from typing import Tuple
import torch
import numpy as np
from torch.utils.data import DataLoader, random_split
import torchvision as tv
from torchvision import transforms

MNIST_MEAN, MNIST_STD = (0.1307,), (0.3081,)
CIFAR_MEAN, CIFAR_STD = (0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010)

def set_seed(seed: int = 42, deterministic_cudnn: bool = True) -> None:
    import random
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    if deterministic_cudnn:
        torch.backends.cudnn.deterministic = True
        torch.backends.cudnn.benchmark = False

def _make_loaders(train_ds, val_ds, test_ds, batch_size: int, num_workers: int):
    pin = torch.cuda.is_available()
    train_loader = DataLoader(train_ds, batch_size=batch_size, shuffle=True,
                              num_workers=num_workers, pin_memory=pin,
                              persistent_workers=(num_workers > 0))
    val_loader = DataLoader(val_ds, batch_size=batch_size, shuffle=False,
                            num_workers=num_workers, pin_memory=pin,
                            persistent_workers=(num_workers > 0))
    test_loader = DataLoader(test_ds, batch_size=batch_size, shuffle=False,
                              num_workers=num_workers, pin_memory=pin,
                              persistent_workers=(num_workers > 0))
    return train_loader, val_loader, test_loader

def get_loaders(dataset: str = "mnist",
                 batch_size: int = 128,
                 seed: int = 42,
                 num_workers: int = 2):
    """
    Return no-augmentation loaders and metadata:
    (train_loader, val_loader, test_loader, in_ch, num_classes, in_shape)
    """
    set_seed(seed)
    g = torch.Generator().manual_seed(seed)
    name = dataset.lower()

    if name not in {"mnist", "cifar10", "cifar-10"}:
        raise ValueError("dataset must be 'mnist' or 'cifar10'")

    if name == "mnist":
        tf = transforms.Compose([
            transforms.ToTensor(),
            transforms.Normalize(MNIST_MEAN, MNIST_STD),
        ])
        full = tv.datasets.MNIST(root="data", train=True, download=True, transform=tf)
        train_ds, val_ds = random_split(full, [50_000, 10_000], generator=g)
        test_ds = tv.datasets.MNIST(root="data", train=False, download=True, transform=tf)
        in_ch, num_classes, in_shape = 1, 10, (1, 28, 28)
    else: # CIFAR-10
        tf = transforms.Compose([
            transforms.ToTensor(),
            transforms.Normalize(CIFAR_MEAN, CIFAR_STD),
        ])
        full = tv.datasets.CIFAR10(root="data", train=True, download=True, transform=tf)
        train_ds, val_ds = random_split(full, [45_000, 5_000], generator=g)
        test_ds = tv.datasets.CIFAR10(root="data", train=False, download=True, transform=tf)
        in_ch, num_classes, in_shape = 3, 10, (3, 32, 32)

    train_loader, val_loader, test_loader = _make_loaders(
        train_ds, val_ds, test_ds, batch_size=batch_size, num_workers=num_workers
    )
    return train_loader, val_loader, test_loader, in_ch, num_classes, in_shape
```

Usage

```
python Copy code

train_l, val_l, test_l, in_ch, n_classes, in_shape = get_loaders("cifar10", batch
```

This mirrors your earlier API, just without any augmentation hooks, only dataset-level

```

train_loader, val_loader, test_loader = _make_loaders(
    train_ds, val_ds, test_ds, batch_size=batch_size, num_workers=num_workers
)
return train_loader, val_loader, test_loader, in_ch, num_classes, in_shape

```

Usage

```

python                                     Copy code

train_l, val_l, test_l, in_ch, n_classes, in_shape = get_loaders("cifar10", batch

```

This mirrors your earlier API, just without any augmentation knobs—only dataset-level normalization for stable training.

📄 👍 🗨️ ⬆️ ↺ ⋮

my cifar runs are giving a matrix multiplication size error

Thought for 13s >

That error almost always means your first `nn.Linear` is getting the wrong input size. For CIFAR-10, each image is $3 \times 32 \times 32 = 3072$ features when flattened. If your MLP was built with `in_dim=784` (MNIST), you'll see a message like:

```

java                                     Copy code

RuntimeError: mat1 and mat2 shapes cannot be multiplied (64x3072 and 784x128)

```

Quick fix (MLP)

Make sure you build the MLP with the **right `in_dim` from the loaders**:

```

python                                     Copy code

# After get_loaders(...)
train_l, val_l, test_l, in_ch, num_classes, in_shape = get_loaders(dataset="cifar
in_dim = in_shape[0] * in_shape[1] * in_shape[2] # 3*32*32 = 3072
model = MLP(in_dim=in_dim, hidden_sizes=[512,256,128], num_classes=num_classes, p

```

And in your `MLP.forward`, keep the flatten:

```

python                                     Copy code

def forward(self, x):
    x = x.view(x.size(0), -1) # or torch.flatten(x, 1)
    return self.net(x)

```

Quick sanity print (to confirm)

Run these once to verify shapes match:

```

python                                     Copy code

xb, yb = next(iter(train_l))
print("batch:", xb.shape) # expect [8, 3, 32, 32] for CIFAR-10
print("flatten to:", xb.view(xb.size(0), -1).shape) # [8, 3072]
print("MLP first layer expects:", model.net[0].in_features) # should be 3072

```

Other common causes (and fixes)

- **Wrong dataset stats/transform** that changes size: ensure your CIFAR transform leaves images 32×32 (no random crop to smaller without `resize`).
- **Accidentally using MNIST loaders/config** when `dataset='cifar10'`. Double-check the `dataset` arg and that you're not hardcoding `in_dim=784`.
- **CNN case only**: if you try a CNN head without `AdaptiveAvgPool2d(1)`, be careful: MNIST becomes 7×7 after two pools, CIFAR becomes 8×8 . Using GAP avoids mismatches.

If you paste your `model` construction line and the exact error text (the two shapes), I'll point to the exact field to change.

📄 👍 🗨️ ⬆️ ↺ ⋮



```
↳ import torch.nn as nn class CNN2(nn.Module): def __init__(self, in_ch=3, num_classes=10,
base=32, enhanced=False, deeper=False, p_drop=0.5): super().__init__() bn = (lambda c:
nn.BatchNorm2d(c)) if enhanced else (lambda c: nn.Identity()) self.features = nn.Sequential(...
```

can you rewrite this class but without accounting for padding

Thought for 13s >

Here's a padding-free rewrite of your `CNN2` that never uses `padding=1` and also **doesn't hard-code** any spatial sizes. It uses `AdaptiveAvgPool2d(1)` so you don't have to compute `fc_in` by hand.

```
python Copy code

import torch
import torch.nn as nn

class CNN2_NoPad(nn.Module):
    """
    Same idea as your CNN2, but:
    - All convs are 3x3 with *no padding* (spatial dims shrink).
    - Uses AdaptiveAvgPool2d(1) so the FC head is size-agnostic.
    - Keeps the 'enhanced' (BatchNorm/Dropout) and 'deeper' options.
    """
    def __init__(self, in_ch=3, num_classes=10, base=32,
                  enhanced=False, deeper=False, p_drop=0.5):
        super().__init__()
        bn = (lambda c: nn.BatchNorm2d(c)) if enhanced else (lambda c: nn.Identity())

        # Feature extractor: conv3x3 (no padding) → ReLU → 2x2 maxpool
        blocks = [
            nn.Conv2d(in_ch, base, kernel_size=3, padding=0, bias=not enhanced),
            bn(base), nn.ReLU(inplace=True),
            nn.MaxPool2d(2),

            nn.Conv2d(base, base*2, kernel_size=3, padding=0, bias=not enhanced),
            bn(base*2), nn.ReLU(inplace=True),
            nn.MaxPool2d(2),
        ]

        if deeper:
            blocks += [
                nn.Conv2d(base*2, base*4, kernel_size=3, padding=0, bias=not enhanced),
                bn(base*4), nn.ReLU(inplace=True),
                nn.MaxPool2d(2),
            ]

        self.features = nn.Sequential(*blocks)

        out_ch = base*4 if deeper else base*2

        # Head: global average to 1x1 → flatten → FC
        self.classifier = nn.Sequential(
            nn.AdaptiveAvgPool2d(1),
            nn.Flatten(),
            nn.Dropout(p_drop) if enhanced else nn.Identity(),
            nn.Linear(out_ch, 128),
            nn.ReLU(inplace=True),
            nn.Linear(128, num_classes),
        )

    def forward(self, x):
        x = self.features(x)
        x = self.classifier(x)
        return x
```

Notes (so it "just works")

- No padding $\Rightarrow$ spatial dims shrink each conv by 2 pixels per side. Pooling halves them. For typical inputs it stays valid:
 - MNIST 28×28 , `deeper=False`: $28 \rightarrow 26 \rightarrow 13 \rightarrow 11 \rightarrow 5$ ✓
`deeper=True`: $5 \rightarrow 3 \rightarrow 1$ ✓
 - CIFAR-10 32×32 , `deeper=False`: $32 \rightarrow 30 \rightarrow 15 \rightarrow 13 \rightarrow 6$ ✓
`deeper=True`: $6 \rightarrow 4 \rightarrow 2$ ✓
- Because we use `AdaptiveAvgPool2d(1)`, the FC in-features are just `out_ch` regardless of the final $H \times W$ —no manual 7×7 vs 8×8 math and no dataset-specific branches.
- Keep eval/training modes (`model.eval()` vs `model.train()`) correct when you toggle `enhanced=True` (BatchNorm/Dropout behave differently).

can you write a class for the baseline cnn as seen in the report

Thought for 11s >

Here's a simple **baseline CNN** that matches the report's shape:

Conv(32) → ReLU → MaxPool → Conv(64) → ReLU → MaxPool → FC(128) → ReLU → FC(num_classes)

(No BatchNorm, no Dropout, padding=1 to preserve H×W before pooling.)

```
python Copy code

import torch
import torch.nn as nn

class BaselineCNN(nn.Module):
    """
    Baseline: 2 conv blocks + 2x2 pools, then 2 FC layers.
    Works for MNIST (1x28x28) and CIFAR-10 (3x32x32).
    """
    def __init__(self, in_ch: int, num_classes: int, input_hw: tuple[int,int], ba
        super().__init__()
        H, W = input_hw

        # Feature extractor
        self.features = nn.Sequential(
            nn.Conv2d(in_ch, base, kernel_size=3, padding=1), # -> base channels
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2), # H/2, W/2

            nn.Conv2d(base, base*2, kernel_size=3, padding=1), # -> 2*base channe
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2), # H/4, W/4
        )

        # Infer the flattened size automatically from input shape
        with torch.no_grad():
            dummy = torch.zeros(1, in_ch, H, W)
            feat = self.features(dummy)
            fc_in = feat.numel() # 1 * C * H' * W'

        # Classifier head
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(fc_in, 128),
            nn.ReLU(inplace=True),
            nn.Linear(128, num_classes),
        )

    def forward(self, x):
        x = self.features(x)
        x = self.classifier(x)
        return x
```

How to use it

```
python Copy code

# From your loader: in_shape = (C, H, W)
train_l, val_l, test_l, in_ch, num_classes, in_shape = get_loaders("cifar10", bat
model = BaselineCNN(in_ch=in_ch, num_classes=num_classes, input_hw=(in_shape[1],
```

- For MNIST, `in_shape = (1, 28, 28)` → pools to 7×7 before the FC.
- For CIFAR-10, `in_shape = (3, 32, 32)` → pools to 8×8 before the FC.

The dummy pass auto-computes the correct `fc_in` so you don't have to hardcode sizes.